QueryGenie: Confidence-Aware Abstention for Open-Weight Text-to-SQL

1st N V V S S Jayakanth Kamisetti
Dept. of Computer Science and IT
KL University
Hyderabad, India
2320090050

2nd Boddupally Narendar
Dept. of Computer Science and IT
KL University
Hyderabad, India
2320090009

3rd M Praneeth Reddy
Dept. of Computer Science and IT
KL University
Hyderabad, India
2320090018

Supervisor: M Parameswar
Associate Professor, Dept. of Computer Science and IT
KL University
Hyderabad, India

***Abstract*—Open-weight text-to-SQL models such as CodeS return one query per question and give no indication of whether it can be trusted. We reproduce the open CodeS-1B model, then make it say when it is unsure. On the 1,034 Spider development questions we measure 77.8% execution accuracy against the paper's 77.9%, using the authors' own evaluator. We execute the four candidate queries that beam search already produces, count how many return the same result, and abstain when that agreement is low. At 90.4% coverage the system is right 83.3% of the time instead of 77.8%, and gives 156 wrong answers instead of 230; over ten calibration/test splits the gain over random abstention is $+5.46$ points (paired t-test $p = 7.7\times10^{-10}$, Wilcoxon $p = 9.8 \times 10^{-4}$, bootstrap 95% CI $[+4.5, +6.7]$). Because beam candidates are often copies, we measure how many are different (all four in 56% of questions) and repeat the study over different queries only, with CodeS-3B, and with eight sampled candidates; the gain holds in each. The pipeline reaches 69.5% on Spider-Realistic (paper: 70.9%) and runs on a live PostgreSQL database through a read-only SQL translation, where 94.4% of the Spider gold queries return the same rows as on SQLite. It ships as a desktop application that runs locally; code and logs are at github.com/jayakanthh/QueryGenie. Test-suite accuracy and a full BIRD evaluation are TBD.**

***Keywords*—text-to-SQL, selective prediction, abstention, execution agreement, reproducibility, CodeS, Spider**

I. Introduction

A natural-language interface to a database is only useful if its users can tell when to believe it. Text-to-SQL models are usually compared by how often they are right when they answer every question [1], [2], but a wrong query is more costly than no query: it returns a plausible table that looks like an answer. Many high-scoring systems rely on closed hosted models [3]–[5], which means a company's schema is sent to a third party. CodeS [6] offers open models of 1B to 15B parameters that score as well on Spider, but it returns one query per question with uniform apparent confidence. Confidence estimation and abstention for semantic parsing and text-to-SQL have been studied [7]–[10], including consistency over candidates and execution results [10], [11]; we do not claim execution agreement as a new idea. Our contribution is to measure it carefully on a reproduced open model and to put it into a working local application.

This paper does three things.

1) **Reproduction.** We reproduce SFT CodeS-1B on Spider [12] within 0.2 points of the published figure (77.76% against 77.9%), after auditing our harness line by line against the authors' code and fixing the errors that audit found (Section III).
2) **Confidence-aware abstention.** We use the candidates CodeS already generates (no extra model or training): the system executes all of them, measures how many agree on a result, and declines to answer, with a clarifying choice, when agreement is low. We evaluate this as an accuracy–coverage trade-off with a pre-registered significance protocol (Section IV).
3) **Candidate diversity and other settings.** We test the claim behind the method — that agreement among candidates carries information — when the candidates are partly duplicates, with a larger model, with sampled candidates, on rewritten questions and on a live PostgreSQL server (Sections V–VI).

All numbers below come from logged runs (seed 42, hardware and runtime recorded for each) and the scripts that analyse them; a number we did not measure is marked TBD. The code, run logs and analysis scripts are in the project repository, https://github.com/jayakanthh/QueryGenie.

II. Related Work

Spider [12] is the standard cross-domain benchmark, scored today by execution rather than by string match, with test-suite accuracy [13] as a stricter variant. Spider-Realistic [14], Spider-Syn [15] and BIRD [16] probe robustness to wording, synonyms and large real-world databases. Methods moved from graph encoders [17], [18] and constrained decoding [19] to large language models, prompted [3], [4], [20], [21] or fine-tuned [6], [22]. Value matching in the style of BRIDGE [23], which CodeS keeps, adds matched cell values to the prompt.

Confidence and abstention. Our rule combines two older ideas: self-consistency, which trusts the answer that most sampled reasoning paths reach [24], and selective classification,

which lets a model decline inputs and reports the accuracy–coverage trade-off [25]. For semantic parsing, Stengel-Eskin and Van Durme study confidence estimation [7] and, in "Did You Mean...?", use calibrated confidence to decide when to ask the user for help, trading safety against convenience [26], the same trade-off our application faces. For text-to-SQL, Somov and Tutubalina [8] add selective classifiers and study entropy-based confidence and its coverage–risk trade-off; Maleki et al. [10] compare black-box and white-box confidence estimates for generated SQL and find consistency-based methods the strongest black-box choice, with execution adding signal; Liu et al. [27] calibrate confidence from sub-clause frequencies over several samples; TrustSQL [9] and EHRSQL [28] score systems on answering feasible questions and abstaining on infeasible ones; and Chen et al. [29] abstain at the schema-linking step. Execution results have also been used to rerank candidate programs [30], to pick the most execution-consistent query [11], and in DAIL-SQL's voting [4]. Recent studies compare confidence signals for generated SQL [31] and show that the risk a conformal abstention rule reports depends on the execution oracle that labels correctness [32]. Questions with several valid queries are the subject of AmbiQT [33] and AMBROSIA [34], which is the situation in which our application shows the competing readings. Against this background our contribution is narrower: a reproduced open 1B model, agreement over the candidates it already produces, a pre-registered significance protocol, a check of the signal against duplicated candidates and other settings, and an application that offers the user the competing readings when it abstains.

Repair. Execution-guided repair, in the spirit of DIN-SQL's self-correction [3], addresses queries that fail; we measure it and find that it addresses a small share of CodeS's errors (Section IV).

III. Experimental Setup and Reproduction

Data. All data come from approved public sources on Hugging Face (Table I). Spider's development databases are small (2 to 11 tables, 7 to 56 columns) and its questions often name the columns; 39.5% of the gold queries need a join, 26.8% a `GROUP BY`, 8.0% a nested query and 7.7% a set operation. By the official difficulty classifier the 1,034 questions are 248 easy, 446 medium, 174 hard and 166 extra hard.

Pipeline. For each question CodeS builds a prompt from the database: every table with its columns, types, keys and representative values, human-readable column descriptions, and stored values matched to words in the question by the authors' value retriever. The model generates four candidates by beam search and returns the first that SQLite accepts (`EXPLAIN QUERY PLAN`), otherwise the first beam. We use the authors' fine-tuned `seeklhy/codes-1b-spider` (and `codes-3b-spider`), fp16.

Harness audit. Our first runs scored 64.8–73.8%. We compared the harness line by line with the authors' code and found a bug in our port (numbers were never matched by the value retriever, changing the prompt for 12.4% of questions), missing column descriptions, and a simplified scoring metric that was not the paper's. We fixed these, switched to the official evaluator [13] (execution match), and kept the earlier runs in the log, marked as superseded.

TABLE I. Evaluation sets and setup. All runs use seed 42.

Spider dev	1,034 questions, 20 databases (reproduction, ablation, abstention)
Spider-Realistic	508 questions, 19 databases (rewritten wording)
BIRD Mini-Dev	500 questions, 11 databases (stopped, not reported)
Metric	official Spider execution match (`evaluation.py --etype exec`)
Decoding	4 beams, at most 256 new tokens, prompts cut to 3,840 tokens
Hardware	Colab Tesla T4, fp16 (CodeS-1B, 3B, sampling); Apple M4, bf16 (Spider-Realistic, application)
Software	PyTorch 2.11 and transformers 5.16 (T4); PyTorch 2.13 and transformers 5.15 (M4)
Run time	2,865 s for one Spider dev pass of CodeS-1B (T4); 5,543 s for CodeS-3B

TABLE II. Execution accuracy (EX) on Spider dev, 1,034 questions.

Run	Setting	EX
T2-001	no value retriever, simple metric	64.8%
T2-003	full prompt, simple metric	73.8%
T2-005	**audited harness, official metric**	**77.8%**
	paper, CodeS-1B	*77.9%*
T2-004	CodeS-3B, official metric	81.8%
	paper, CodeS-3B	*83.4%*

Result. The corrected pipeline reaches 77.8% execution accuracy (804/1034; run T2-005, re-run from scratch as T2-006 with an identical result), against 77.9% in the paper. Table II shows the effect of each fix and of the model size. The 3B model reaches 81.8% (paper: 83.4%, -1.6 points). The paper also reports test-suite accuracy of 72.2% for CodeS-1B; we did not compute ours (TBD).

Ablation. Removing one component at a time (official metric, same model and seed) lowers accuracy in every case (Table III). The differences are 9–20 questions each and were not tested for significance, so the order of the two smallest effects should not be over-read. Accuracy falls with difficulty: 90.3% on easy, 83.6% medium, 64.9% hard and 56.6% extra-hard questions.

Deviations. We match values by scanning all distinct values rather than the authors' Lucene BM25 pre-filter; we do not apply the optional schema classifier; and we use newer PyTorch and transformers versions on one T4 instead of eight A800 GPUs.

IV. Confidence-Aware Abstention

Method. Of the 97.3% of queries that execute, many still return the wrong rows, so repairing failures cannot fix most errors. Execution-guided repair, which returns each failed query to the model once together with its error message, was tried on 377 failed queries over 211 questions; 61 then ran and one more answer became correct (77.76% $\rightarrow$ 77.85%). We therefore turned to the queries that run but are wrong. All

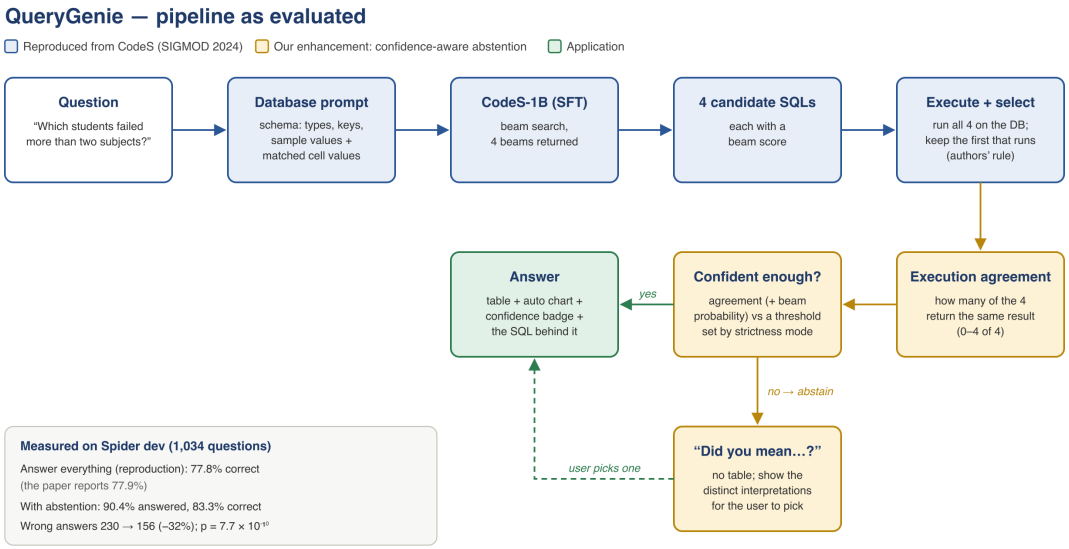


Fig. 1. System architecture: the reproduced CodeS components, the abstention step added here, and the application.

TABLE III. Ablation on Spider dev (CodeS-1B, official metric).

Run	Component removed	EX	Δ	Executable
T2-005	none (full pipeline)	77.8%	—	97.3%
AB-105	foreign keys	76.9%	−0.9	95.9%
AB-103	column values	76.4%	−1.4	98.1%
AB-106	selection (beam 0 only)	76.0%	−1.8	94.1%
AB-104	value retriever	75.9%	−1.9	97.2%

Algorithm 1 Answer or ask, for one question

Require: question Q, database D, threshold τ
$P \leftarrow$ CodeS database prompt for (Q, D)
$(s_1, \ldots, s_4), (b_1, \ldots, b_4) \leftarrow$ four beam candidates and scores
for $i = 1$ **to** 4 **do**
  $r_i \leftarrow$ result of running s_i on D (read-only; none if it fails)
end for
$k \leftarrow$ first i whose query passes `EXPLAIN QUERY PLAN`, else 1
$a \leftarrow \frac{1}{4}|\{i : r_i = r_k\}|$ if s_k runs, else 0
$c \leftarrow a + 0.01\, e^{b_k} / \sum_j e^{b_j}$
if $c \geq \tau$ **then**
  return result r_k and agreement a
else
  return the distinct results among $r_1, \ldots, r_4$ as readings to choose from
end if

candidates are executed (read-only, on the original database); the *agreement* a of the returned answer is the share of candidates whose result equals it, and 0 if the returned query fails. The confidence is $c = a + 0.01\, p_b$, where p_b is the softmax beam probability of the returned candidate (a tie-breaker only). The system abstains when $c < \tau$: it then shows no table, but lists the distinct results of the candidates as readings for the user to choose from (Algorithm 1, Fig. 1). No model is retrained.

Protocol. Fixed before the results were seen: ten random 50/50 calibration/test splits of the 1,034 questions; τ chosen on the calibration half for at least 90% coverage and applied to the test half; accuracy on answered questions compared with random abstention at the same coverage (the model's accuracy on that half); one-sided paired t-test and Wilcoxon signed-rank test at $\alpha = 0.05$; and a question-level bootstrap (5,000 resamples) that does not rely on the splits being independent. The protocol was written in the Week-9 enhancement proposal. The deployed threshold, $\tau = 0.2526$, is the median over the ten calibration halves. In words: abstain when no candidate runs, or when only one of four supports the answer and its beam probability is below about 0.26.

TABLE IV. Answering everything vs. abstaining (CodeS-1B, run T2-006).

Setting	Answered	Acc. on answered	Wrong
Answer everything	1,034 (100%)	77.76%	230
Balanced ($\tau = 0.2526$)	**935 (90.4%)**	**83.32%**	**156**
Strict ($\geq$3 of 4 agree)	649 (62.8%)	86.13%	90
Very strict (all 4 agree)	412 (39.8%)	88.35%	48

Results. Table IV and Fig. 2 show the outcome. At 90.4% coverage the system answers correctly 83.3% of the time against 77.8% for answering everything (this full-set figure applies the deployed τ to the questions it was calibrated on; the held-out mean over the ten splits is 83.25% at 90.5% coverage); wrong answers fall from 230 to 156 (−32%). Of the 99 questions it declines, 74 would have been wrong and 25 correct. The mean gain over random abstention is +5.46 points (SD 0.71), positive in all ten splits: paired $t = 24.4$, $p = 7.7 \times 10^{-10}$; Wilcoxon $W = 55$, $p = 9.8 \times 10^{-4}$ (the smallest value possible with ten pairs); bootstrap 95% CI $[+4.51, +6.71]$. We reject the null hypothesis of no gain. Beam probability alone gives a smaller gain (+3.08 points); agreement is higher than beam probability by 2.37 points on the same splits (paired $t = 6.31$, $p = 1.4 \times 10^{-4}$).

Why it works, and where it does not. Accuracy rises

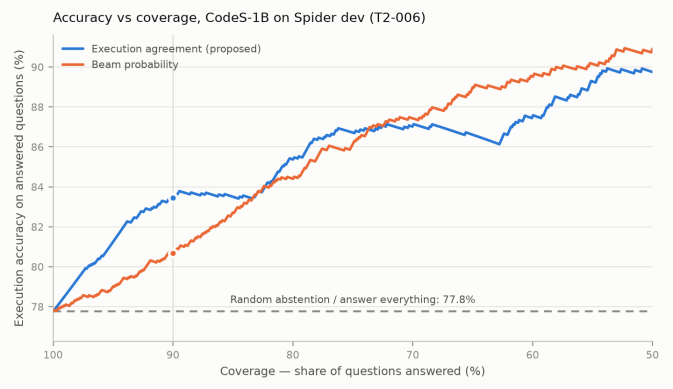


Fig. 2. Accuracy on answered questions against coverage (CodeS-1B, Spider dev).

TABLE V. Accuracy by number of candidates (of 4) that agree with the returned answer (CodeS-1B, run T2-006).

Agreeing candidates	0	1	2	3	4
Questions	28	132	225	237	412
Accuracy	0.0%	56.8%	75.6%	82.3%	88.4%

steadily with the number of candidates that agree (Table V): when the answer's result is produced by no other candidate it is right 56.8% of the time, and when all four agree 88.4%. When the returned query does not run at all the answer is never right. Agreement is not calibrated at the top, however: 11.7% of answers on which all four candidates agree are still wrong, because beam candidates share the model's blind spots. Over the whole curve beam probability ranks answers slightly better (area under the risk–coverage curve 0.0905 against 0.1025 for agreement; random 0.2224, perfect ranking 0.0269); agreement is ahead from full coverage down to about 74% coverage, the region where the operating point lies. Agreement takes only five values, so intermediate coverages rely on the beam-probability tie-breaker.

V. Are the Candidates Really Different?

A natural objection is that beam search often returns the same query several times, in which case agreement among the candidates would mean little, and a related question is why the number of candidates is four. This was also raised at a review of this work, so we measured it. We re-executed every saved candidate of run T2-006.

Duplicates. Here two candidates are the same query if they differ only in spacing or capitals (so alias renamings count as different, which makes this a lower bound on duplication). The four beams are four different queries for only 577 of 1,034 questions (55.8%); 42 questions have one distinct query, 190 two and 225 three. For CodeS-3B the counts are 57, 234, 249 and 494.

Agreement over different queries. We recomputed agreement ("v2") counting each different query once and comparing results up to row and column order, which also corrects 68 questions where only the column order hid agreement. Over different queries, accuracy rises with the number that agree: 0 agree: 0.0% (28 questions); 1: 71.7% (244); 2: 78.7% (367); 3: 83.1% (207); 4: 89.4% (188). Table VI shows that the abstention gain is unchanged in kind: $+5.14$ points as a share and $+5.66$ as a count, against $+5.46$ for the evaluated rule. The conclusion does not depend on duplicates.

Another model. For CodeS-3B agreement gives $+3.18$ points (89.7% coverage, 85.2% accuracy on answered) and, over different queries, $+3.88$. Abstention helps the larger and the smaller model, by less for the larger one, which is already right more often. For CodeS-3B, however, beam probability is as good a signal as agreement ($+3.44$ points), so the advantage of agreement over beam probability seen for CodeS-1B does not carry over (Fig. 3, left).

Sampled candidates. We also drew eight candidates by sampling (temperature 0.8) instead of beams (run SM-008). Plain accuracy is 77.08% (beams: 77.76%); a correct query is among the eight for 881 questions (85.2%) against 859 (83.1%) among four beams. Majority vote over the eight reaches 77.66%, whereas majority vote over four beams was worse than the first-executable rule (75.73% against 77.76%). Agreement abstention holds (Table VI, Fig. 3, right), and the 491 questions on which all eight agree are correct 93.5% of the time.

Why four? Four is what CodeS's inference code returns and what our threshold was calibrated on. The application now lets the user choose 2–12 candidates, over-generating beams (or sampling at rising temperature until the queries differ) so that candidates are different queries wherever the model has more than one way to answer. This was done after the evaluation: the measured accuracy and threshold are for four beams, and other counts are uncalibrated. A run with eight beams printed 78.8% accuracy before it failed while saving its results; it is being re-run, and no further number from it is reported (TBD).

VI. Beyond Spider

Rewritten questions. Spider questions often name the columns. On the 508 development questions that Spider-Realistic [14] rewrites without column names, the pipeline (run on an Apple M4) reaches 69.5% (353/508) against the paper's 70.9% (-1.4 points). On the original wording of the same 508 questions it reaches 72.8% (370/508), so the wording costs 3.3 points (bootstrap 95% CI $[0.8, 5.9]$; 31 questions right only in the original wording, 14 only in the realistic one; exact McNemar $p = 0.016$). On the same 508 questions the M4 run chose the same SQL as the T4 run on 485, and accuracy was 72.8% against 73.2%. Agreement stays informative (Table VII): accuracy is 86.9% when all four candidates agree and 47.1% when one does. A run on BIRD Mini-Dev [16] was stopped after 18 of 500 questions and is not reported (TBD).

A live database. CodeS writes SQLite. For a server database we translate each query with sqlglot, accept only one read-only statement, and run it in a read-only transaction with a timeout and a row limit; the schema and a value index are read once at connection time. We copied the 20 Spider development databases into PostgreSQL 14 and ran every gold query and every saved prediction on both engines (Table VIII). The final rewrites (string literals, case-insensitive `LIKE`, aggregate grouping) bring agreement between the engines from about

TABLE VI. Abstention signals compared (mean of ten calibration/test splits; gain over random abstention at the same coverage; paired t-test p; Wilcoxon $p = 9.8 \times 10^{-4}$ in every row). AURC: area under the risk–coverage curve, lower is better. Beam probability and the count variant are not meaningful for sampled candidates (repeats are signal there) and are omitted.

Run (model, candidates)	Signal	Coverage	Acc. answered	Gain	p (t-test)	AURC
T2-006 (CodeS-1B, 4 beams)	agreement (evaluated)	90.5%	83.3%	+5.46	7.7×10^{-10}	0.103
	agreement over different queries (share)	90.6%	82.9%	+5.14	1.7×10^{-9}	0.101
	agreement over different queries (count)	90.0%	83.5%	+5.66	4×10^{-11}	0.122
	beam probability	89.7%	80.9%	+3.08	9.3×10^{-7}	0.091
T2-004 (CodeS-3B, 4 beams)	agreement (evaluated)	89.7%	85.2%	+3.18	2.9×10^{-6}	0.078
	agreement over different queries (share)	89.1%	85.9%	+3.88	3.6×10^{-7}	0.072
	agreement over different queries (count)	89.4%	85.9%	+3.95	1×10^{-7}	0.091
	beam probability	88.5%	85.4%	+3.44	2.9×10^{-7}	0.065
SM-008 (CodeS-1B, 8 samples)	agreement (evaluated)	90.0%	82.5%	+5.57	1.7×10^{-7}	0.109
	agreement over different queries (share)	89.5%	83.1%	+6.19	1.5×10^{-8}	0.102

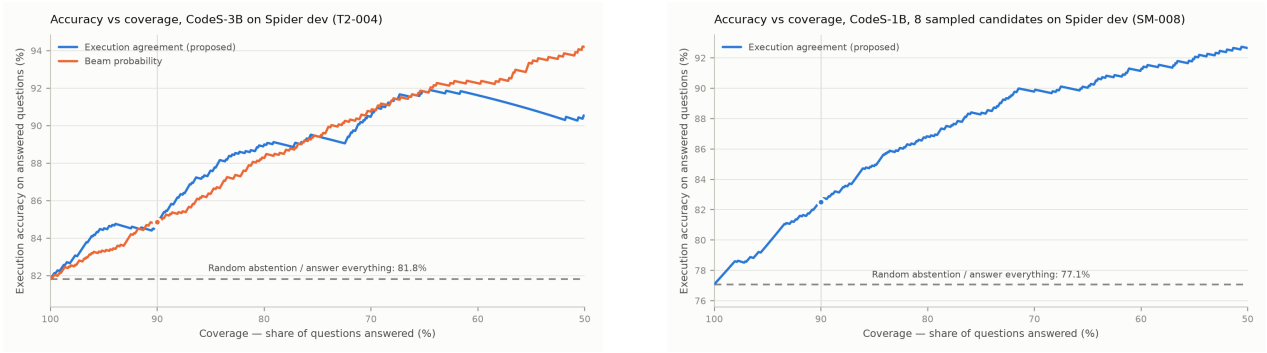


Fig. 3. Accuracy against coverage for CodeS-3B with four beams (left) and CodeS-1B with eight sampled candidates (right). Beam probability is not drawn for sampled candidates.

TABLE VII. Original and Spider-Realistic wording of the same 508 questions (CodeS-1B).

	Original	Realistic
EX, answering everything	72.8% (370)	69.5% (353)
Balanced: questions answered	88.6%	86.2%
Balanced: accuracy on answered	79.3%	76.7%
Wrong answers, all → balanced	138 → 93	155 → 102

TABLE VIII. Queries returning the same rows on PostgreSQL as on SQLite.

	Gold queries	Model predictions
Plain translation	67.0%	71.1%
Final rewrites	**94.4% (976/1034)**	**93.7% (943/1006)**
different rows / errors	41 / 17	49 / 14

two thirds to about 94%; most remaining mismatches are `ORDER BY...LIMIT` queries with tied rows, which legitimately differ between engines. End-to-end accuracy and calibration on PostgreSQL, and MySQL, are not measured (TBD); a qualitative run of twelve questions on one database matched the gold answer ten times.

VII. The Application

The system ships as a desktop application (macOS; a Windows build is configured but untested) that runs the same evaluated code as the experiments (Fig. 4). Every answer shows how many of the candidate queries agree; the "all queries" view runs and displays every candidate with its own result, marks copies, and shows the repair of a failed query. The user chooses how strict the system is (answer everything, balanced, strict, very strict), with the measured coverage and accuracy of Table IV shown next to each choice, and when the system abstains it lists the competing readings. Databases can be the built-in samples, a SQLite file or a live PostgreSQL or MySQL server; a saved server connection keeps its password in the operating system's keychain, never in a file. The model is the evaluated CodeS-1B by default, and local CodeS variants, Ollama models and hosted services can be chosen; only CodeS-1B with four beams has measured accuracy, and a hosted service receives the schema, a few example values and the question (result rows only if the user opts in to sharing earlier rows for follow-ups). Follow-up questions use the earlier questions of a chat as context for chat models only, because CodeS was trained on single questions. The core package has 55 model-free tests and is released under the Apache-2.0 licence.

VIII. Discussion and Threats to Validity

What the results say. The model's own candidates carry usable information about its reliability: declining the 9.6% least-agreed questions removes 74 wrong answers at the price of 25 correct ones (3.1% of the 804 the model gets right), and the three strictness settings give the user a dial between coverage and accuracy. The gain holds when duplicates are

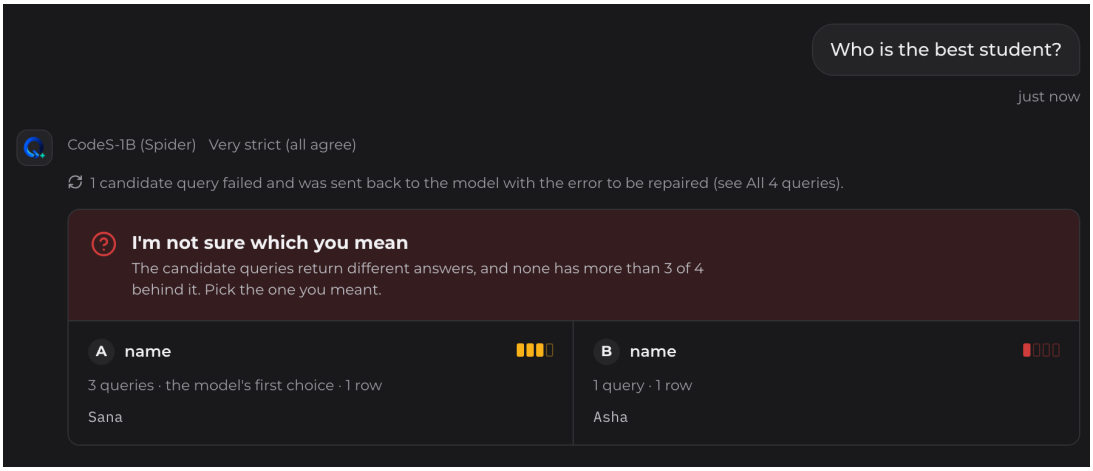


Fig. 4. The application when it is not sure (very strict setting). Three of the four candidate queries return one result (reading A) and one returns another (reading B); that is not enough agreement, so the system lists the readings with their support instead of answering. One candidate failed and was repaired, as the note above the card says.

removed, for a second model size and for sampled candidates, so it is not an artefact of repeated beams. It is also modest and bounded: unanimous candidates are still wrong 11.7% of the time, and for the 3B model a cheaper signal, beam probability, is as good.

Internal validity. The deployed threshold was chosen on the same development set it is reported on; the split protocol and bootstrap guard against over-fitting to a particular half but not against the set itself. Each configuration was run once with seed 42; beam search is deterministic (a repeat of one run was identical), but the sampled run has not been repeated with other seeds. The Colab and M4 runs agree on 485 of 508 selected queries, so hardware has a small effect. **Construct validity.** Correctness is Spider's execution match on one database instance per schema; a stricter oracle such as test-suite accuracy can change the labels and so the measured risk [32], and we have not computed it. **External validity.** Spider's databases are small and clean; the Spider-Realistic gap of 3.3 points shows sensitivity to wording, and BIRD, the larger real-world benchmark, is TBD, so we make no claim about enterprise databases.

Limitations. (1) One benchmark and, for the main result, one model. (2) The ten splits reuse the same 1,034 questions. (3) The ablation differences are small and untested for significance. (4) Test-suite accuracy, BIRD, MySQL and end-to-end PostgreSQL accuracy are TBD, and the larger CodeS models were not run. (5) Candidate counts other than four, and models other than CodeS-1B, are uncalibrated. **Responsible use.** The default setup runs on the user's machine with open weights and read-only database access, so no schema or data leave it; choosing a hosted model sends the schema to its provider, and the application says so.

IX. Conclusion and Future Work

An open 1B-parameter text-to-SQL model can be reproduced to within 0.2 points of its published accuracy and made to decline the questions it is likely to get wrong: at 90.4% coverage it is right 83.3% of the time instead of 77.8%, with a third fewer wrong answers, using only the candidates it already generates and no retraining. The result survives duplicated candidates, a larger model and sampled candidates, and the system works on rewritten questions and a live server database. Next steps are the results marked TBD (test-suite accuracy, BIRD, the eight-beam run, MySQL and end-to-end PostgreSQL accuracy), calibrating the threshold for other candidate counts, per-database adaptation, and role-based access when connected to a company database.

Acknowledgment

The authors thank M Parameswar for supervision, and the Department of Computer Science & Information Technology, KL University, for the project framework.

References

[1] N. Deng, Y. Chen, and Y. Zhang, "Recent Advances in Text-to-SQL: A Survey of What We Have and What We Expect," *arXiv preprint arXiv:2208.10099*, 2022, COLING 2022.

[2] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, "Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL," *arXiv preprint arXiv:2406.08426*, 2024, IEEE TKDE 2025.

[3] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," *arXiv preprint arXiv:2304.11015*, 2023, NeurIPS 2023.

[4] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," *arXiv preprint arXiv:2308.15363*, 2023, PVLDB 2024.

[5] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, and S. O. Arik, "CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL," *arXiv preprint arXiv:2410.01943*, 2024.

[6] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, "CodeS: Towards Building Open-source Language Models for Text-to-SQL," *arXiv preprint arXiv:2402.16347*, 2024, SIGMOD 2024.

[7] E. Stengel-Eskin and B. Van Durme, "Calibrated Interpretation: Confidence Estimation in Semantic Parsing," *arXiv preprint arXiv:2211.07443*, 2022, TACL 2023.

[8] O. Somov and E. Tutubalina, "Confidence Estimation for Error Detection in Text-to-SQL Systems," *arXiv preprint arXiv:2501.09527*, 2025, AAAI 2025.

[9] G. Lee, W. Chay, S. Cho, and E. Choi, "TrustSQL: Benchmarking Text-to-SQL Reliability with Penalty-Based Scoring," *arXiv preprint arXiv:2403.15879*, 2024.

[10] S. E. Maleki, M. Pourreza, and D. Rafiei, "Confidence Estimation for Text-to-SQL in Large Language Models," *arXiv preprint arXiv:2508.14056*, 2025.

[11] Ł. Borchmann and M. Wydmuch, "Query and Conquer: Execution-Guided SQL Generation," *arXiv preprint arXiv:2503.24364*, 2025.

[12] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," *arXiv preprint arXiv:1809.08887*, 2018, EMNLP 2018.

[13] R. Zhong, T. Yu, and D. Klein, "Semantic Evaluation for Text-to-SQL with Distilled Test Suites," *arXiv preprint arXiv:2010.02840*, 2020, EMNLP 2020.

[14] X. Deng, A. H. Awadallah, C. Meek, O. Polozov, H. Sun, and M. Richardson, "Structure-Grounded Pretraining for Text-to-SQL," *arXiv preprint arXiv:2010.12773*, 2020, NAACL 2021.

[15] Y. Gan, X. Chen, Q. Huang, M. Purver, J. R. Woodward, J. Xie, and P. Huang, "Towards Robustness of Text-to-SQL Models against Synonym Substitution," *arXiv preprint arXiv:2106.01065*, 2021, ACL 2021.

[16] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, X. Zhou *et al.*, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," *arXiv preprint arXiv:2305.03111*, 2023, NeurIPS 2023.

[17] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," *arXiv preprint arXiv:1911.04942*, 2019, ACL 2020.

[18] R. Cao, L. Chen, Z. Chen, Y. Zhao, S. Zhu, and K. Yu, "LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and Non-Local Relations," *arXiv preprint arXiv:2106.01093*, 2021, ACL 2021.

[19] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," *arXiv preprint arXiv:2109.05093*, 2021, EMNLP 2021.

[20] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, and Z. Li, "MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL," *arXiv preprint arXiv:2312.11242*, 2023, COLING 2025.

[21] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual Harnessing for Efficient SQL Synthesis," *arXiv preprint arXiv:2405.16755*, 2024.

[22] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL," *arXiv preprint arXiv:2302.05965*, 2023, AAAI 2023.

[23] X. V. Lin, R. Socher, and C. Xiong, "Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing," *arXiv preprint arXiv:2012.12627*, 2020, Findings of EMNLP 2020.

[24] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," *arXiv preprint arXiv:2203.11171*, 2022, ICLR 2023.

[25] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," *arXiv preprint arXiv:1705.08500*, 2017, NeurIPS 2017.

[26] E. Stengel-Eskin and B. Van Durme, "Did You Mean...? Confidence-based Trade-offs in Semantic Parsing," *arXiv preprint arXiv:2303.16857*, 2023, EMNLP 2023.

[27] T. Liu, S. Wang, D. Preotiuc-Pietro, Y. Chandarana, and C. Gupta, "Calibrating LLMs for Text-to-SQL Parsing by Leveraging Sub-clause Frequencies," *arXiv preprint arXiv:2505.23804*, 2025, EMNLP 2025.

[28] G. Lee, H. Hwang, S. Bae, Y. Kwon, W. Shin, S. Yang, M. Seo, J.-Y. Kim, and E. Choi, "EHRSQL: A Practical Text-to-SQL Benchmark for Electronic Health Records," *arXiv preprint arXiv:2301.07695*, 2023, NeurIPS 2022 Datasets and Benchmarks.

[29] K. Chen, Y. Chen, X. Yu, and N. Koudas, "Reliable Text-to-SQL with Adaptive Abstention," *arXiv preprint arXiv:2501.10858*, 2025.

[30] A. Ni, S. Iyer, D. Radev, V. Stoyanov, W. Yih, S. I. Wang, and X. V. Lin, "LEVER: Learning to Verify Language-to-Code Generation with Execution," *arXiv preprint arXiv:2302.08468*, 2023, ICML 2023.

[31] R. Richardson, "What Predicts Correctness in Text-to-SQL? A Selective-Prediction Study," *arXiv preprint arXiv:2607.06799*, 2026.

[32] J. Liu, D. Qiao, Y. Zhang, and X. Chen, "Certified Against Which Oracle? Execution Labels Set the Reported Risk of Conformal Abstention for Text-to-SQL," *arXiv preprint arXiv:2609.25938*, 2026.

[33] A. Bhaskar, T. Tomar, A. Sathe, and S. Sarawagi, "Benchmarking and Improving Text-to-SQL Generation under Ambiguity," *arXiv preprint arXiv:2310.13659*, 2023, EMNLP 2023.

[34] I. Saparina and M. Lapata, "AMBROSIA: A Benchmark for Parsing Ambiguous Questions into Database Queries," *arXiv preprint arXiv:2406.19073*, 2024, NeurIPS 2024 Datasets and Benchmarks.